# Datasets

## Samples, Splits & Batching

**Thanh-Sach LE**

✉ ltsach@hcmut.edu.vn

Faculty of Computer Science and Engineering
HCMUT, VNU-HCM

Ho Chi Minh City on February 5, 2026



Outline

Datasets

Data Quality & Preprocessing

Splits & Evaluation Setup

Batching & Training Loop

Imbalanced Data & Augmentation

Benchmarks & Data Sources

Dataset & DataLoader in Practice

# Outline

1. Datasets
2. Data Quality & Preprocessing
3. Splits & Evaluation Setup
4. Batching & Training Loop
5. Imbalanced Data & Augmentation
6. Benchmarks & Data Sources
7. Dataset & DataLoader in Practice

# Datasets



Outline
2 Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# What is a Dataset?

## Definition

A dataset is a collection of input–output pairs:

$$\mathcal{D} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{N}$$

- $\boldsymbol{x}_i$: input feature vector (column vector)
- $\boldsymbol{y}_i$: target (label)
- $N$: number of samples

## Key View

Machine learning = learning a function

$$f : \boldsymbol{x} \to \boldsymbol{y}$$

from data.



Outline
3 Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Sample, Feature, and Label

## Single Sample

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \in \mathbb{R}^{d \times 1}$$

- Feature: one measurable attribute
- Sample: one data point
- Label: ground-truth output



Outline
4 Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Regression Dataset

## Form

$$\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$$

- Target is scalar

$$y_i \in \mathbb{R}$$

- Example: house price, temperature, age



Outline
5 Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Classification Dataset



## Form

$$\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$$

- Target is class label

$$y_i \in \{1, \ldots, K\}$$

- Binary: $K = 2$
- Multiclass: $K > 2$

Outline
6 Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Types of Data in ML & Deep Learning

## Beyond tabular vectors

- **Tabular:** $\boldsymbol{x}_i \in \mathbb{R}^d$ (features), standard in classical ML
- **Images:** $\boldsymbol{x}_i \in \mathbb{R}^{C \times H \times W}$ (channels, height, width)
- **Sequences:** $\boldsymbol{x}_i = (x_1, \ldots, x_T)$ (text, time series, audio)
- **Graphs:** nodes, edges, node/edge features (GNNs)

## Supervised vs unsupervised

Labels $\boldsymbol{y}_i$ may be absent: clustering, self-supervised learning, generative models.



Outline
Datasets 7
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Deep Learning: Image — Sample & Batch Shape

## Single image (one sample)

$$\mathbf{x}_i \in \mathbb{R}^{C \times H \times W}$$

- $C$: channels (e.g. 3 for RGB), $H$: height, $W$: width
- PyTorch convention: **NCHW** (channel-first)

## Batch of images

$$X_{\text{batch}} \in \mathbb{R}^{B \times C \times H \times W}$$

$B$ images stacked along the first dimension; GPU ops expect 4D tensors.

- Example: CIFAR-10 batch $\Rightarrow$ shape `(32, 3, 32, 32)`

# Deep Learning: Text — Sample & Batch Shape

## Single sequence (one sample)

- Token IDs: $\boldsymbol{x}_i \in \mathbb{R}^{T}$ (length $T$; variable across samples)
- After embedding: $\boldsymbol{x}_i \in \mathbb{R}^{T \times D}$ with $D =$ embedding dim

## Batch of sequences

Typically **pad** to same length $T_{\text{max}}$:

$$X_{\text{batch}} \in \mathbb{R}^{B \times T_{\text{max}}} \text{ (token IDs)} \quad \text{or} \quad \mathbb{R}^{B \times T_{\text{max}} \times D} \text{ (embedded)}$$

Use a **attention mask** or **lengths** to ignore padding in the loss.




Outline
9 Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Deep Learning: Time-Series — Sample & Batch Shape

## Single time series (one sample)

$$\boldsymbol{x}_i \in \mathbb{R}^{T \times F}$$

- $T$: time steps (length of the sequence)
- $F$: number of features per time step (e.g. sensor channels, univariate $F = 1$)

## Batch of time series

$$X_{\text{batch}} \in \mathbb{R}^{B \times T \times F}$$

Same as text: pad to fixed $T$ if lengths differ, then mask in the model.



Outline
10 Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Tabular (Vector) Data: Matrix View

For **feature vectors** $\boldsymbol{x}_i \in \mathbb{R}^d$, stack row-wise:

$$X = \begin{bmatrix} \boldsymbol{x}_1^\top \\ \boldsymbol{x}_2^\top \\ \vdots \\ \boldsymbol{x}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times d}$$

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$$

### Why Rows?

Matches PyTorch / Keras: each row = one sample; batches are contiguous rows.



Outline
11 Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Data Quality & Preprocessing



Outline
Datasets
12 Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Data Quality: What Can Go Wrong?

- **Missing values:** NaNs, blanks → impute (mean, median, model) or drop
- **Noise & outliers:** measurement error, rare events → robust scaling, clipping, or removal
- **Label noise:** wrong labels → hurts supervised learning; use robust losses or cleaning
- **Duplicate samples:** inflate performance → deduplicate before splitting
- **Data bias:** underrepresentation of groups → fairness and generalization concerns



Outline
Datasets
13 Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Preprocessing: Normalization & Encoding

## Normalization (e.g. zero mean, unit variance)

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

Fit $\mu_j$, $\sigma_j$ *only on the training set*; then apply same transform to validation/test.

## Categorical features

Encode as one-hot or embeddings; again, fit encoder on training data only.

## Rule

Any statistic used in preprocessing must be computed from the training set to avoid data leakage.

# Splits & Evaluation Setup



Outline
Datasets
Data Quality & Preprocessing
15 Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Train / Validation / Test Split

- **Training set:** fit model parameters
- **Validation set:** tune hyperparameters, early stopping, model selection
- **Test set:** final evaluation *once*; report metrics here

Typical split: e.g. 70%–15%–15% or 80%–10%–10% (depends on $N$).



Outline
Datasets
Data Quality & Preprocessing
16 Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Stratified Split (Classification)

For classification, keep class proportions similar in train/val/test.

**Stratified split**

Each split has roughly the same fraction of each class as the full dataset.

- Prevents a rare class from missing in validation or test
- Use `StratifiedKFold`, `stratify=` in `train_test_split` (sklearn) or equivalent in PyTorch



Outline
Datasets
Data Quality & Preprocessing
17 Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Cross-Validation

When data is limited, use $K$-fold cross-validation:

- Partition training data into $K$ folds
- For each fold: train on $K - 1$ folds, validate on the remaining fold
- Average validation metrics over $K$ runs; reduces variance of the estimate

**Note**

Test set is still held out; CV is used for hyperparameter tuning or model selection on the *training* part only.



Outline
Datasets
Data Quality & Preprocessing
18 Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Time Series & Sequential Data

For temporal data, do **not** shuffle then split: use a temporal split.

- Train: earlier time steps; validation: next period; test: future period
- Prevents future information from leaking into training



Outline
Datasets
Data Quality & Preprocessing
19 Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Batching & Training Loop



Outline

Datasets

Data Quality & Preprocessing

Splits & Evaluation Setup

20 Batching & Training Loop

Imbalanced Data & Augmentation

Benchmarks & Data Sources

Dataset & DataLoader in Practice

# Mini-batch

Instead of full dataset:

$$X_b \in \mathbb{R}^{B \times d}$$

- **Faster:** gradient over $B$ samples instead of $N$
- **Generalization:** noise in mini-batch gradients can act as regularization
- **SGD:** standard optimization in deep learning; batch size is a key hyperparameter



Outline
Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
21 Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Shuffling

- Randomly permute samples each epoch
- Prevents model from learning order patterns
- Improves convergence



Outline
Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
22 Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Data Leakage

**Definition**

Information from test set leaks into training.

Examples:

- Normalize using whole dataset
- Augment after splitting incorrectly

**Rule**

Test set must be untouched until the final evaluation.



Outline
Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
23 Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice



Faculty of Computer Science and Engineering
HCMUT, VNU-HCM

# Imbalanced Data & Augmentation

Datasets

Outline

Datasets

Data Quality & Preprocessing

Splits & Evaluation Setup

Batching & Training Loop

24 Imbalanced Data & Augmentation

Benchmarks & Data Sources

Dataset & DataLoader in Practice

# Class Imbalance

When one or more classes have many fewer samples than others:

- Model may predict the majority class most of the time; accuracy can be misleading
- Use metrics: Precision, Recall, F1, PR curve, ROC-AUC (see Metrics slides)
- **Data-level:** oversample minority (e.g. SMOTE), undersample majority, or both
- **Algorithm-level:** class weights in the loss, focal loss, threshold tuning



Outline
Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
25 Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Data Augmentation (Deep Learning)

Increase effective dataset size by generating variants of training samples.

### Images

Crop, flip, rotate, color jitter, cutout, MixUp, etc. Applied *on the fly* during training (different each epoch).

### Text / sequences

Back-translation, synonym replacement, random insert/delete (careful: preserve labels).

### Rule

Augment only the training set; validation/test stay original for fair evaluation.



Outline
Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
26 Imbalanced Data & Augmentation
Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Benchmarks & Data Sources



Outline

Datasets

Data Quality & Preprocessing

Splits & Evaluation Setup

Batching & Training Loop

Imbalanced Data & Augmentation

27 Benchmarks & Data Sources

Dataset & DataLoader in Practice

Faculty of Computer Science and Engineering
HCMUT, VNU-HCM

# Common Benchmarks

- **Vision:** MNIST, CIFAR-10/100, ImageNet, COCO (detection/segmentation)
- **Tabular:** UCI ML Repository, Kaggle datasets
- **Text:** GLUE, SQuAD, WikiText
- **Recommendation / graphs:** MovieLens, OGB

Use standard splits and metrics so results are comparable across papers and implementations.



Outline
Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
28 Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Where to Get Data

- Public repositories: UCI, Kaggle, Hugging Face Datasets, PyTorch/TF datasets
- APIs and crawling (respect terms of use and copyright)
- Synthetic data when real data is scarce or sensitive
- Always document source, license, and preprocessing for reproducibility



Outline
Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
29 Benchmarks & Data Sources
Dataset & DataLoader in Practice

# Dataset & DataLoader in Practice

Outline
Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
30 Dataset & DataLoader in Practice

# Dataset vs Dataloader



## Dataset (e.g. `torch.utils.data.Dataset`)

- Implements `__len__` and `__getitem__`
- Stores or loads samples and labels; can apply transforms when returning a sample

## DataLoader

- Mini-batching (collate samples into a batch tensor)
- Shuffling (each epoch)
- Parallel loading (`num_workers`) and optional `pin_memory` for GPU

Outline
Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
31 Dataset & DataLoader in Practice

# DataLoader: Key Parameters

- **batch_size:** trade-off speed vs memory and gradient noise
- **num_workers:** parallel data loading; $0 =$ main process only
- **pin_memory:** can speed up CPU$\rightarrow$GPU transfer when using CUDA
- **drop_last:** drop last incomplete batch so each epoch has same number of updates (sometimes useful)



Outline
Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
32 Dataset & DataLoader in Practice

# Summary

- Dataset = samples + labels; types: tabular, images, sequences, graphs
- **DL shapes:** image $(B, C, H, W)$; text/time-series $(B, T)$ or $(B, T, F)$ with padding & mask
- Preprocessing (normalization, encoding) fit on *training* data only; avoid leakage
- Train/val/test split; use stratified split for classification; time-order split for sequences
- Cross-validation for model selection when data is limited
- Handle imbalance: metrics, resampling, or class weights
- Augmentation on training data only; standard benchmarks enable comparison
- Dataset implements `__getitem__`; DataLoader handles batching, shuffling, workers



Outline
Datasets
Data Quality & Preprocessing
Splits & Evaluation Setup
Batching & Training Loop
Imbalanced Data & Augmentation
Benchmarks & Data Sources
33 Dataset & DataLoader in Practice